

COMP3230 Programming Assignment Two

Performance Analysis Report

Student Number: xxxxxxxxxx
Platform: Apple M2 (8 cores), Docker Ubuntu Linux

November 12, 2025

1 Benchmark Results

Table 1 summarizes elapsed execution times across different configurations.

Size	Seq (s)	2T (s)	4T (s)	8T (s)
200×200	0.89	1.57	1.84	3.02
400×400	8.87	7.46	6.07	7.88
600×600	33.37	20.26	14.24	17.56
800×800	96.13	44.46	32.92	39.97
1000×1000	—	85.24	62.01	74.96

Table 1: Elapsed times (bold = best parallel performance per size).

2 Analysis

2.1 Problem Size Impact

Small problems (200×200) suffer from parallelization overhead. Sequential execution (0.89s) outperforms even 2-thread parallel (1.57s) by 76%. With only 39,204 interior points and 30,628 iterations, per-iteration work is insufficient to amortize synchronization costs.

Larger problems show clear benefits. Speedup factors relative to sequential: 400×400 achieves 1.46×, 600×600 achieves 2.34×, and 800×800 reaches 2.92×. The trend demonstrates that computational intensity eventually dominates overhead when problem size increases.

2.2 Thread Count and Synchronization

Four threads consistently deliver optimal performance despite 8 available cores. The 800×800 case illustrates the pattern: 2T (44.46s), 4T (32.92s), 8T (39.97s), 16T (46.09s). Performance degrades beyond 4 threads due to barrier synchronization overhead.

The Jacobi method requires all threads to synchronize after each iteration. With 155,611 iterations for 800×800, this creates 155,611 barrier points. Each barrier forces early-finishing threads to wait idle for stragglers. As thread count increases:

- More threads increase probability of scheduling delays affecting barrier completion

- Context switching overhead rises (1.69M voluntary switches observed for 800×800 8T)
- Cache coherence traffic increases with more threads accessing shared data

Resource usage statistics reveal the cost: 800×800 8T configuration shows worker threads spending 0.44–0.49s in system time (kernel synchronization operations) out of 12s total per-thread time.

The fine-grained nature of Jacobi iterations exacerbates this. Each iteration performs minimal computation per point (4 reads, 3 adds, 1 division, 1 write 10–15 cycles). Even microsecond barrier latencies accumulate significantly across 100,000+ iterations, preventing linear speedup.

2.3 Optimal Configuration

The barrier synchronization pattern creates a fundamental scalability ceiling. Thread count beyond physical core count (or half of it, accounting for hyperthreading effects) introduces more overhead than benefit. The 4-thread sweet spot on this 8-core M2 system likely reflects optimal mapping to performance cores without triggering excessive context switching or cache contention.

3 Conclusion

Effective parallelization of the Jacobi method requires: (1) problem sizes 600×600 to overcome synchronization overhead, (2) thread counts matching available performance cores (4 threads optimal here), and (3) recognition that per-iteration barriers fundamentally limit scalability. On the test platform, 4 threads with large grids achieved up to 2.92× speedup, while smaller problems or excessive thread counts degraded performance below sequential baseline.